TripSync

Wspólny wyjazd. Jeden plan.

Dokumentacja projektu

Aplikacja Android do wspólnego planowania i rozliczania wyjazdów.

Wersja aplikacji: 1.2.1 / kod wersji 4. Backend API: 1.2.0.

Data dokumentacji: 06.11.2026.

Wykonawcy

Lp.	Imię i nazwisko
1	Brajan Sieroń
2	Damian Panońko
3	Patryk Dawczak

Obszar	Rozwiązanie
Platforma	Kotlin, Compose / Material 3, MVVM, Android 8.0+
Dane i integracje	Room, REST, MySQL lub SQLite, Open-Meteo i OpenStreetMap
Testy z 06.11.2026	99 testów zaliczonych; lint: 0 błędów, 1 ostrzeżeń
Repozytorium	https://github.com/Yamoteusz/TripSync
Aplikacja	TripSync.apk w paczce TripSync_do_oddania.zip

Dokument obejmuje cel i zakres, wymagania, architekturę, przypadki użycia, model danych, technologie, instalację, wyniki testów oraz najważniejsze widoki aplikacji.

Spis treści

01. Cel, zakres i wymagania
02. Funkcje i korzystanie z aplikacji
03. Zmiany w wersji 1.2.1
04. Architektura i synchronizacja
05. Przypadki użycia
06. Bazy danych i model informacji
07. Biblioteki i technologie
08. API REST - kontrakt backendu 1.2.0
09. Instalacja i uruchomienie
10. Bezpieczeństwo i obsługa błędów
11. Testowanie i wyniki wersji 1.2.1
12. Ograniczenia aplikacji
13. Materiały do oddania - TripSync
14. Zrzuty najważniejszych widoków

Informacje o wydaniu

Aplikacja: Android 1.2.1. Backend: API 1.2.0. Minimalna wersja systemu: Android 8.0 / API 26. Projekt używa compile SDK 36.1 i target SDK 36.

Data testów i zrzutów ekranu: 06.11.2026. Środowisko urządzeniowe: emulator Android 10 / API 29. Szczegółowy zakres, wyniki oraz ograniczenia testów opisuje rozdział testowania.

Repozytorium kodu źródłowego: https://github.com/Yamoteusz/TripSync.

Cel, zakres i wymagania

Cel projektu

TripSync porządkuje organizację podróży niewielkich grup znajomych: plan, zadania, pomysły i wspólne koszty. Aplikacja umożliwia wspólne ustalanie harmonogramu, dzielenie obowiązków i rozliczanie wydatków, również przy okresowym braku dostępu do sieci. Wersja aplikacji: Android 1.2.1; backend API 1.2.0.

Role i odbiorcy

Gość może utworzyć konto, zalogować się lub skorzystać z demo. Uczestnik współpracuje przy planie, zadaniach, pomysłach, miejscach i kosztach. Organizator ma dodatkowe uprawnienia do zarządzania wyjazdem, zaproszeniami, uczestnikami i budżetem. System Android wykonuje zadania WorkManager i lokalne powiadomienia. Uprawnienia opisuje macierz w **przypadki-uzycia.md**.

Wymagania funkcjonalne

ID	Wymaganie	Kryterium odbioru
F01	Rejestracja, logowanie, wylogowanie	Walidacja, sesja lokalna i unieważnienie tokenu przy wylogowaniu online
F02	CRUD wyjazdów	Nazwa, cel, daty, waluta i opis; edycja/usunięcie przez organizatora
F03	Uczestnicy i zaproszenia	Kod 12 znaków lub link; rotacja kodu i kontrola dostępu
F04	CRUD harmonogramu	Data w terminie wyjazdu, godzina, miejsce, kolejność i wykonanie
F05	Propozycje i głosy	CRUD propozycji, jeden własny głos na propozycję, cofnięcie i ponowne oddanie
F06	Propozycja do planu	Jedna wersjonowana operacja zmienia pomysł w punkt, usuwa głosy i nie tworzy duplikatów
F07	Zadania i pakowanie	CRUD, kategoria, osoba, wykonanie i opcjonalne przypomnienie
F08	Wydatki	CRUD, płatnik, data, uczestnicy, podział równy lub kwoty własne
F09	Rozliczenia	Dokładność groszy, suma sald zero i propozycje przelewów
F10	Offline i trwałość	Lokalny zapis bez sieci, ponowny odczyt po uruchomieniu
F11	Synchronizacja	Kolejka, ponowienia bez duplikatów, wersje oraz wybór wersji przy konflikcie
F12	Powiadomienia lokalne	Przypomnienie punktu/zadania, anulowanie po wykonaniu/usunięciu
F13	Stan i ustawienia	Zachowanie formularza po odtworzeniu aktywności, motyw i adres API w DataStore
F14	Obsługa błędów	Walidacja, komunikaty sieci/serwera i stan odrzuconych operacji
F15	Demo i udostępnianie	Edytowalny przykład offline oraz udostępnienie tekstowego planu
F16	Budżet	CRUD limitu i kategorii, limit kategorii 0 i potwierdzanie usunięcia przez organizatora
F17	Kategorie i lista kosztów	Sześć kategorii, zestawienie, filtr, wyszukiwanie i sortowanie
F18	Spłaty	CRUD własnych spłat; salda uwzględniają spłatę częściową i nadpłatę

ID	Wymaganie	Kryterium odbioru
F19	Historia zmian	Autorytatywne wpisy serwera z autorem i czasem, odczyt bez edycji klienta
F20	Miejsca	CRUD współrzędnych, mapa wspólna i otwarcie w zewnętrznych mapach
F21	Wyszukiwanie	Geokodowanie miejscowości i zapis wyniku jako miejsce
F22	Prognoza	7 dni, temperatury, opis oraz szansa opadów
F23	Cache pogody	Ostatni wynik i daty dostępne po błędzie sieci
F24	Daty i godziny	Kalendarz i zegar 24-godzinny; w bazie ISO, prezentacja po polsku
F25	Ochrona formularza	Potwierdzenie odrzucenia zmian w edytorze głównym, budżecie, spłacie i miejscu
F26	Widoki wyjazdów i planu	Wyszukiwanie, aktywne/zakończone wyjazdy, plan według dni

Wymagania niefunkcjonalne

ID	Wymaganie	Realizacja i granice
N01	Material 3 i responsywność	Compose, motywy, przewijanie, maksymalna szerokość i kompaktowe pickery; szeroka weryfikacja tabletów pozostaje do wykonania
N02	Architektura i modularność	MVVM, foldery ui/domain/data i odrębny backend; jedna aplikacja Gradle, nie pełna Clean Architecture
N03	Asynchroniczność	Coroutines, Flow, suspend Room/Retrofit i I/O poza głównym wątkiem
N04	Zmiana konfiguracji	ViewModel i rememberSaveable; test odtworzenia aktywności, bez pełnej macierzy wszystkich konfiguracji
N05	Ochrona danych	Keystore, HTTPS, kontrola dostępu i walidacja; baza Room bez SQLCipher
N06	Dostępność	Etykiety pól i ikon funkcyjnych, kontrast i większa czcionka; audyt TalkBack pozostaje do wykonania
N07	Testy	Domena Kotlin, backend, Compose UI i integracja REST
N08	Rozbudowa	Typowane modele, wersjonowane operacje, repozytorium i eksport schematu Room
N09	Git i dokumentacja	Historia, README, diagramy, testy, screeny, PDF i APK

Powiązanie wymagań z implementacją: zgodnosc.md. Ograniczenia: ograniczenia.md. Wykonawcy: AUTHORS.md.

Funkcje i korzystanie z aplikacji

Początek i konta

Gość wybiera rejestrację, logowanie albo demo. Demo zawiera wyjazd trzech osób; można go edytować offline. Rejestracja na publicznym lub własnym API tworzy odrębne konto. E-mail jest identyfikatorem, bez wysyłania potwierdzenia. Adres API zmienia się przed logowaniem. Wylogowanie czyści lokalną bazę i niewysłane zmiany; kolejne demo odtwarza przykład.

Wyjazdy i plan

Lista umożliwia wyszukiwanie po nazwie i miejscu oraz filtrowanie aktywnych i zakończonych podróży. Organizator ustala daty, miejsce docelowe, opis i walutę. Podsumowanie pokazuje grupę, koszty, zadania i najbliższy przyszły niewykonany punkt według daty i godziny.

Plan jest pogrupowany według dni. Formularz używa kalendarza ograniczonego do terminu wyjazdu i zegara 24-godzinnego. Na kompaktowym ekranie albo przy większej czcionce pojawia się systemowy kalendarz oraz specjalizowany formularz godziny. Punkt może zawierać zapisane miejsce, opis i przypomnienie. Edytor główny oraz formularze budżetu, spłaty i miejsca pytają o odrzucenie niezapisanych zmian przy cofnięciu. Opcja Zostań zachowuje formularz.

Zadania i pomysły

Dwie listy obejmują zadania oraz pakowanie. Element można przypisać osobie i oznaczyć jako wykonany. Pomysły mają tytuł, miejsce i opis. Każdy uczestnik oddaje i cofa własny głos. Przeniesienie propozycji do harmonogramu wymaga potwierdzenia danych punktu w formularzu. Konwersja jest atomowa na serwerze i zachowuje ID; równoczesne przenoszenie przez dwie osoby nie tworzy dwóch punktów. Przypomnienie powstaje od razu również offline.

Wydatki, budżet i spłaty

Wydatek ma kwotę, datę, płatnika i mapę udziałów osób. Podział równy rozdziela pozostałe grosze deterministycznie; podział własny wymaga sumy udziałów równej kosztowi. Kwoty są całkowite w groszach/centach. Nowy wydatek otrzymuje dzisiejszą datę.

Sześć kategorii to transport, noclegi, jedzenie, atrakcje, zakupy i pozostałe. Koszty można wyszukiwać po nazwie, kategorii i płatniku, filtrować i sortować. Filtry i przewinięcie zakładek wracają w bieżącej sesji.

Organizator ustala budżet i limity kategorii. Suma limitów nie może przekroczyć całości. Puste pole kategorii oznacza brak limitu, a wpisane 0 ustala zerowy limit. Usunięcie budżetu wymaga potwierdzenia i pozostawia wydatki. Saldo osoby wynosi: zapłacone wydatki minus jej udziały, plus wysłane spłaty, minus odebrane spłaty. Spłatę zapisuje jej nadawca i tylko on może ją zmieniać lub usunąć. Data nie może pochodzić z przyszłości. To deklaracja, nie transakcja bankowa; nadpłata może odwrócić należność.

Miejsca, mapa i pogoda

Miejsce można dodać ręcznie współrzędnymi lub wybrać miejscowość z wyszukiwarki Open-Meteo. Lista jest współdzielona; na mapie pojawiają się znaczniki i atrybucja OpenStreetMap. Mapa pokazuje ładowanie, błąd i ponowienie. Przejście do zewnętrznych map nie wymaga od aplikacji uprawnienia GPS.

Prognoza obejmuje najbliższe 7 dni dla miejsca: minimum i maksimum temperatury, opis i prawdopodobieństwo opadów. Ostatni wynik jest przechowywany lokalnie i oznaczony czasem pobrania. Nie jest to prognoza na dowolnie odległą datę wyjazdu.

Współpraca i błędy

Organizator udostępnia 12-znakowy kod albo link **tripsync://join?code=KOD**. Pozostali logują się na tym samym API. Zapis offline pozostawia operację w kolejce. Ręczna synchronizacja lub WorkManager wysyła zmiany i pobiera dostępne wyjazdy. Konflikt wersji wymaga wyboru danych lokalnych albo serwera. Historia jest dopisywana przez backend po zaakceptowanej zmianie i pokazuje do 200 najnowszych wpisów.

Usunięcie uczestnika z aktywnymi kosztami/spłatami wymaga wcześniejszego uporządkowania tych danych. Kod zaproszenia można unieważnić przez rotację. Nie ma funkcji zmiany właściciela wyjazdu.

Zmiany w wersji 1.2.1

Wykonawcy: Brajan Sieroń, Damian Panońko, Patryk Dawczak. Data wydania: 06.11.2026. Android: 1.2.1 (versionCode 4), backend: API 1.2.0.

- Atomowe przenoszenie pomysłu w punkt planu: jeden identyfikator, jedna operacja serwera, kontrola wersji, usunięcie głosów i historia w transakcji.
- Ochrona niezapisanych danych w formularzach budżetu, spłaty i miejsca przy cofnięciu strzałką i przyciskiem systemowym.
- Natychmiastowe planowanie przypomnienia po konwersji, także lokalnie i w demo; anulowanie przy odrzuceniu lokalnej konwersji.
- Potwierdzenie usuwania budżetu, z zachowaniem zapisanych wydatków.
- Możliwość zapisania zerowego limitu kategorii przy zachowaniu dodatnich kwot wydatków i spłat.

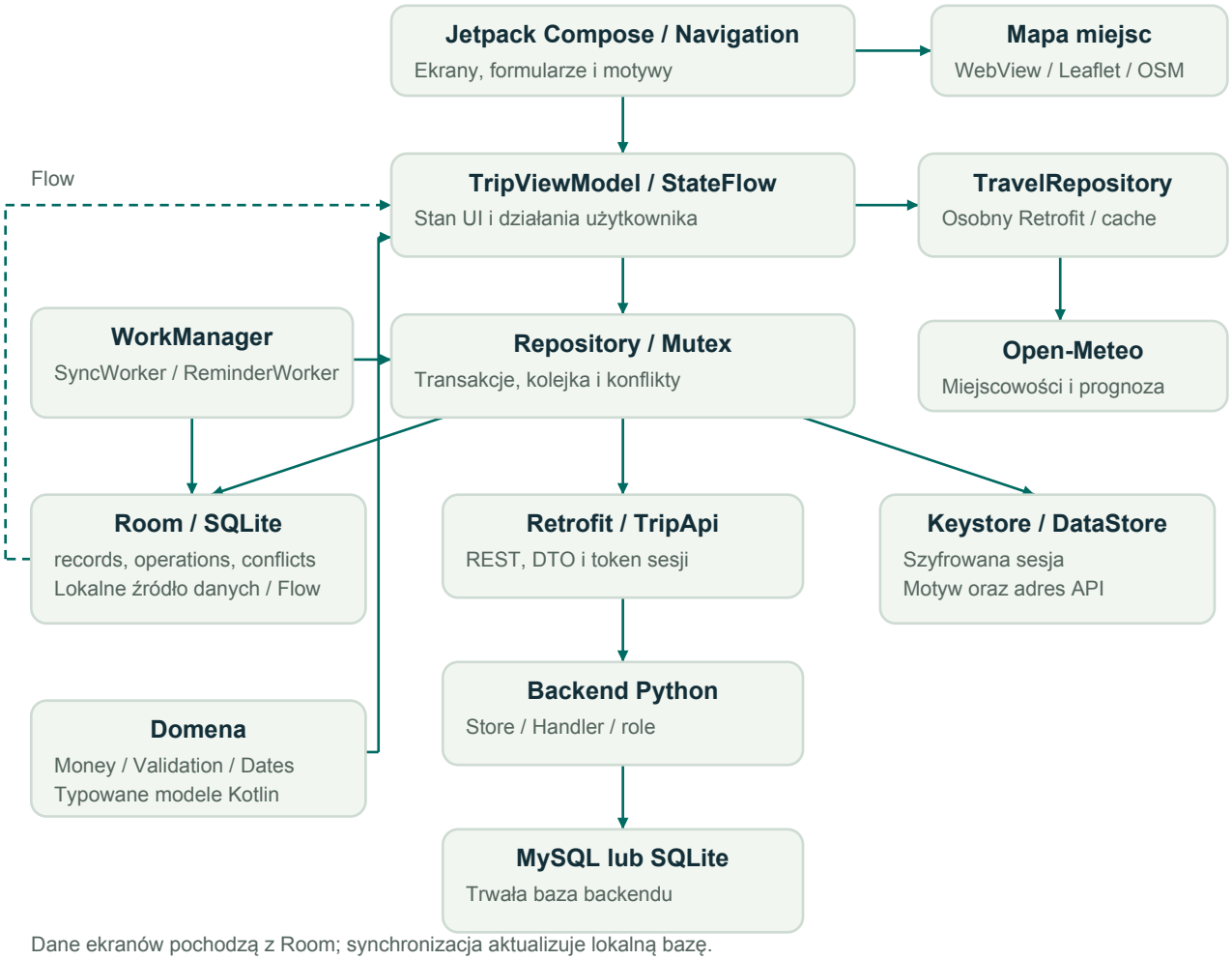
Zakres i wyniki testów opisuje testy.md. Instalacja wymaga aktualizacji backendu przed nowym APK; szczegóły zawiera wispbyte.md.

Architektura i synchronizacja

MVVM

TripSync wykorzystuje MVVM z wydzielonymi folderami domeny i danych. Kod Android jest zorganizowany w jednym module Gradle; backend Python stanowi osobny komponent.

- UI: ekrany Compose, formularze, Theme i Navigation w **ui/**.
- ViewModel: **TripViewModel** zbiera dane repozytorium i publikuje **AppState**, stan błędów, zajętości i pogody przez StateFlow.
- Domena: typowane modele, Money, Validation, Categories i Dates.
- Dane: Room, Repository, TripApi, TravelRepository, Preferences i SessionVault.
- Backend: Handler/Store, autoryzacja, wersjonowanie i adapter SQLite lub MySQL.



Dane ekranów pochodzą z Room; synchronizacja aktualizuje lokalną bazę.

Wersja ilustracyjna: diagram SVG.

Lokalny zapis i przepływ danych

Źródłem danych ekranów jest Room. Formularz waliduje wejście; repozytorium zapisuje rekord oraz operację oczekującą w jednej transakcji. DAO emituje Flow, a ViewModel aktualizuje StateFlow. Ekran zbiera go zgodnie z cyklem życia. Operacje Room/Retrofit są asynchroniczne; pliki i szyfrowana sesja są obsługiwane poza głównym wątkiem.

Mutex repozytorium porządkuje zapisy, logowanie, wylogowanie i synchronizację. Odtworzenie aktywności zachowuje ViewModel i wybrane dane formularza przez rememberSaveable. Dane trwałe pozostają w Room; filtry i przewinięcie zakładek są dodatkowo pamiętane w ViewModel w bieżącej sesji.

Protokół synchronizacji

1. Rekord lokalny i operacja mają identyfikatory, typ, payload i bazową wersję serwera. Ponowna edycja rekordu zastępuje oczekującą operację nowym ID, zachowując bazową wersję.
2. Klient wysyła maksymalnie 100 operacji w batchu, według zależności: wyjazd, propozycja, pozostałe dane, głos.
3. Serwer sprawdza token, dostęp, rolę, dane i **baseVersion**. Poszczególne operacje są odrębnymi transakcjami, nie jedną transakcją całej paczki.
4. Zaakceptowany zapis zwiększa wersję i dopisuje historię w tej samej transakcji. Ponowienie tej samej operacji z tą samą treścią zwraca poprzedni wynik bez duplikatu.
5. Inna wersja bazowa tworzy konflikt. Operacja oczekuje na decyzję o danych lokalnych lub serwera; nie jest automatycznie nadpisywana.
6. Snapshot obejmuje wszystkie dostępne wyjazdy, bez paginacji. Uzupełnia Room i zachowuje niewysłane edycje lokalne.
7. Usunięcie jest przesyłane jako tombstone. Odebranie dostępu usuwa lokalny wyjazd, kolejkę, konflikty i przypomnienia po udanej synchronizacji.

Jeśli odpowiedź serwera zaginie, a klient zmieni rekord ponownie, konflikt może dotyczyć jego własnej wcześniej przyjętej edycji. Mechanizm zachowuje dane, ale wymaga rozstrzygnięcia.

Praca w tle i integracje

SyncWorker ma warunek sieci; okresowe zadanie ma minimalny interwał 15 minut i zależy od Androida. ReminderWorker obsługuje lokalne przypomnienia, bez gwarancji dokładnej godziny.

TravelRepository używa osobnego Retrofit bez tokenu TripSync. Prognoza jest cache'owana lokalnie, a zmiana miejsca anuluje poprzednie żądanie. Pogoda nie trafia do współdzielonej kolejki. Mapa jest własnym HTML w WebView, z lokalnym Leaflet i kafelkami HTTPS.

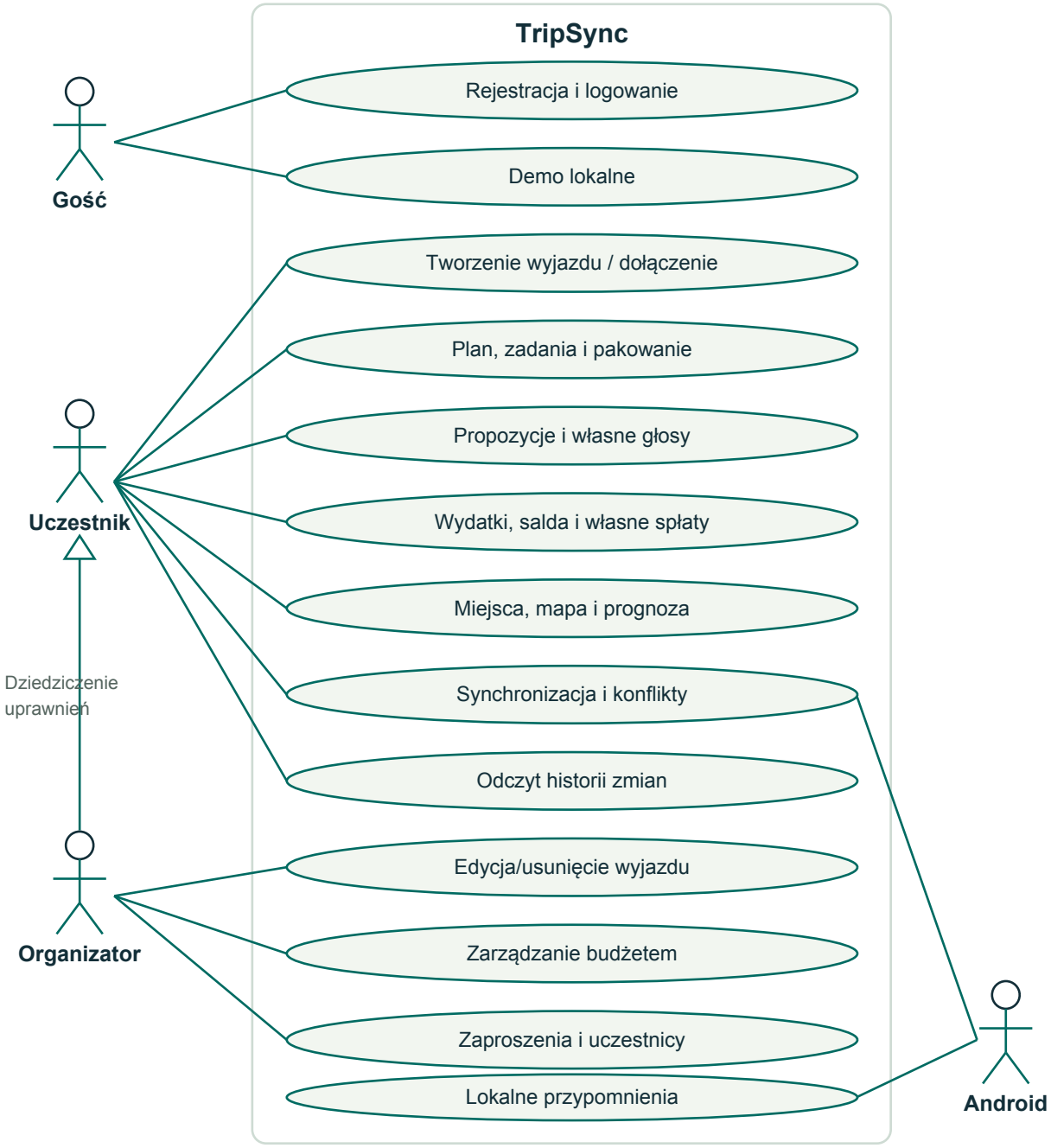
WSGI oraz samodzielny serwer używają wspólnej logiki Store/Handler. Hostowana konfiguracja tworzy Store z adapterem MySQL albo SQLite. Operacje backendu są porządkowane przez blokadę jednego procesu; skalowanie na wiele procesów nie zostało zweryfikowane.

Konwersja pomysłu w wersji 1.2.1

Przeniesienie zachowuje ID rekordu. Room zapisuje lokalny punkt planu i jedną operację **convert** transakcyjnie. Serwer zmienia pomysł w punkt i usuwa głosy w jednej transakcji. Wersjonowanie i stały ID zabezpieczają przed dwoma punktami przy równoczesnej konwersji. Schemat Room pozostaje w wersji 1. Przypomnienie jest planowane natychmiast po lokalnej konwersji, także w demo i offline. Przy przyjęciu wersji serwera wracającej do pomysłu przypomnienie lokalnego punktu jest anulowane.

Przypadki użycia

Diagram UML



Aktorzy znajdują się poza granicą systemu. Organizator jest specjalizacją uczestnika i dziedziczy jego przypadki użycia; dodatkowo zarządza wyjazdem, grupą i budżetem. System Android realizuje zadania w tle. Diagram grupuje powiązane działania CRUD dla czytelności; macierz poniżej precyzuje dostęp.

Macierz uprawnień

Działanie	Gość	Uczestnik	Organizator
Konto / demo	Tak	Po wylogowaniu	Po wylogowaniu

Działanie	Gość	Uczestnik	Organizator
Utworzenie wyjazdu	Nie	Tak, staje się organizatorem nowego wyjazdu	Tak
Edycja/usunięcie istniejącego wyjazdu	Nie	Nie	Tak
Dołączenie kodem/linkiem	Nie	Tak	Tak
CRUD planu, zadań, propozycji, miejsc i kosztów	Tylko lokalne demo	Tak	Tak
Głosowanie	Tylko demo	Za siebie	Za siebie
Spłata: zapis/edycja/usunięcie	Tylko demo	Własna	Własna
Zapis/edycja/usunięcie budżetu	Tylko demo	Nie	Tak
Odczyt budżetu, historii i danych wyjazdu	Tylko demo	Tak	Tak
Kod zaproszenia i jego rotacja	Nie	Nie	Tak
Usunięcie uczestnika	Nie	Nie	Tak, z ograniczeniami finansowymi
Synchronizacja / konflikty	Nie w demo	Tak	Tak

Rola jest przypisana do konkretnego wyjazdu, nie globalnie do konta. Uprawnienia są egzekwowane przez backend; demo działa lokalnie jako organizator przykładu.

UC01 - Utworzenie i udostępnienie wyjazdu

Warunek: użytkownik jest zalogowany. Tworzy wyjazd, wybiera daty oraz walutę, zapisuje i synchronizuje. Backend tworzy członkostwo organizatora. Użytkownik otwiera uczestników i pobiera kod. Druga osoba dołącza, synchronizuje i widzi dane. Nieprawidłowy kod, brak sesji lub sieci daje komunikat; lokalny zapis wyjazdu może oczekiwać w kolejce przed wygenerowaniem zaproszenia.

UC02 - Zapis planu offline

Warunek: wyjazd znajduje się w lokalnej bazie. Użytkownik dodaje punkt z datą w terminie wyjazdu i poprawną godziną. Room zapisuje rekord i operację. Ekran odświeża się bez internetu. Po odzyskaniu sieci synchronizacja przesyła zmianę. Błędne dane są odrzucane; konflikt wersji prowadzi do UC04.

UC03 - Zapis kosztu i spłaty

Uczestnik wpisuje dodatnią kwotę, płatnika i udziały sumujące się do kosztu. Salda aktualizują się lokalnie. Nadawca zapisuje własną spłatę z datą nie późniejszą niż dziś; saldo uwzględnia ten zapis. Nie jest wykonywany przelew bankowy. Edycja cudzej spłaty jest odrzucana na serwerze.

UC04 - Konflikt edycji

Dwie osoby zmieniają ten sam rekord na tej samej wersji bazowej. Serwer przyjmuje pierwszą operację; drugiej zwraca aktualny rekord jako konflikt. Użytkownik wybiera wersję serwera albo ponawia własną na nowej wersji bazowej. Ponowienie może ponownie wejść w konflikt, jeśli nastąpiła kolejna równoczesna zmiana.

UC05 - Mapa i pogoda

Uczestnik zapisuje miejsce lub wybiera miejscowość z wyszukiwarki. Mapa pokazuje znacznik, a prognoza dane dla najbliższych siedmiu dni. Przy braku sieci mapa pokazuje błąd, lista miejsc pozostaje dostępna, a pogoda może wyświetlić oznaczony datą cache. Nie jest pobierany GPS telefonu.

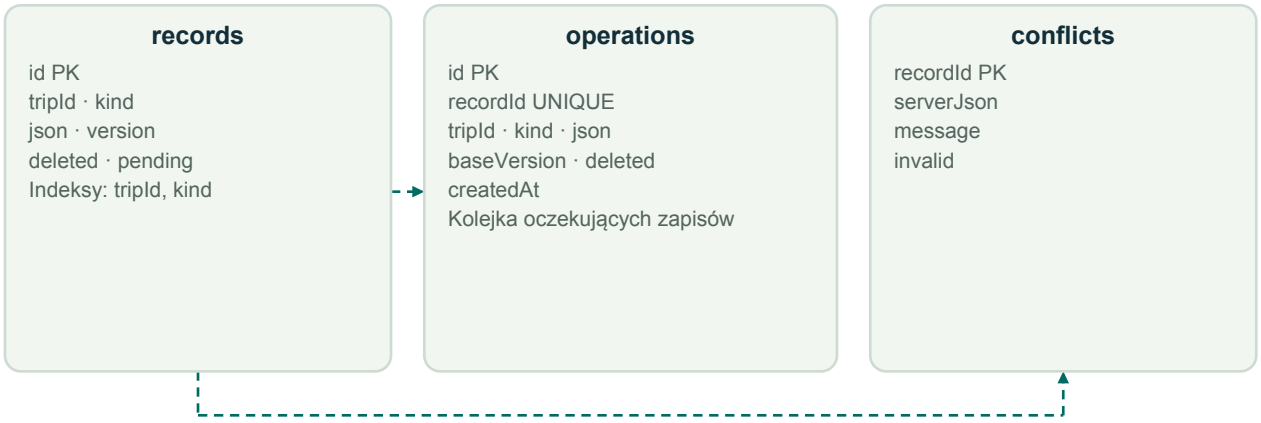
Bazy danych i model informacji

Dlaczego rekordy z payloadem JSON

Wiele typów danych współdzieli identyfikator, wyjazd, wersję i znacznik usunięcia. Jedna tabela rekordów umożliwia jednolity protokół offline i konfliktów. Payloady są reprezentowane typowanymi modelami Kotlin, a spójność referencji jest sprawdzana przez repozytorium i serwer. To świadoma alternatywa wobec osobnej tabeli każdego typu; część relacji nie jest egzekwowana przez FK.

Room na urządzeniu

Schemat wersji 1 znajduje się w **app/schemas/pl.tripsync.data.TripDatabase/1.json**. Wersja Android 1.2.1 nie zmienia fizycznego schematu. Operacja konwersji zachowuje ID i zmienia rodzaj rekordu proposal na event; zapis kolejki używa kind=convert.

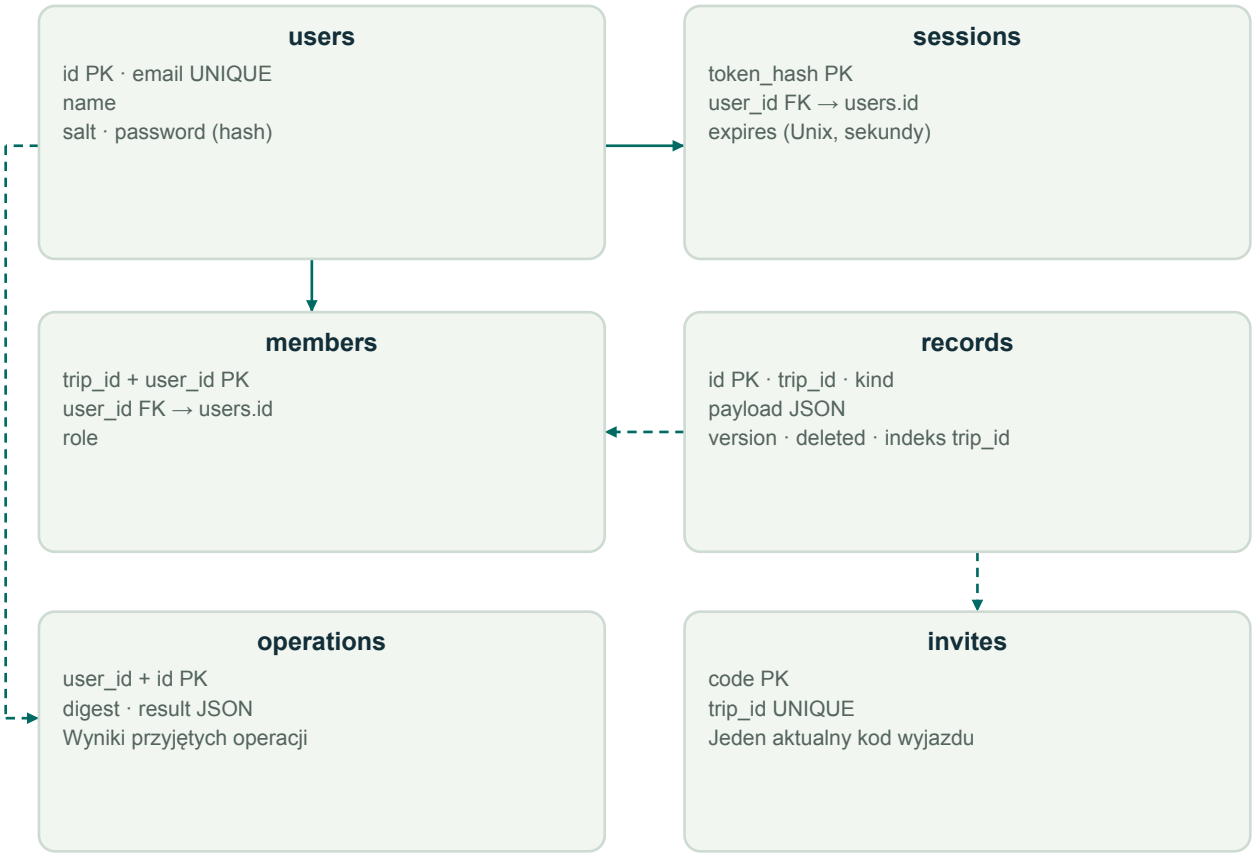


Powiązania logiczne, bez fizycznych FK. Zapis rekordu i operacji w jednej transakcji Room.

Diagram SVG. **records** ma indeksy **tripId** i **kind**. Unikalny indeks **operations.recordId** zapewnia jedną oczekującą operację na rekord. Powiązania diagramu są logiczne; Room nie deklaruje FK pomiędzy tymi trzema tabelami. Zapis rekordu i operacji odbywa się transakcyjnie. Konflikt przechowuje dane serwera i komunikat.

Backend SQLite / MySQL

Oba warianty mają sześć tabel. MySQL używa InnoDB, utf8mb4 i collation utf8mb4_bin; adapter tłumaczy parametry zapytań i operację zastępującą rekord. SQLite włącza FK i WAL. Adapter nie migruje automatycznie istniejącej SQLite do MySQL.



Linia ciągła: fizyczna relacja FK. Linia przerywana: relacja logiczna kontrolowana w kodzie.
MySQL: InnoDB / utf8mb4. SQLite: FK włączone / WAL. Typy szczegółowe: baza-danych.md.

Diagram SVG. Typ **text** na diagramie jest opisem wspólnego modelu; MySQL stosuje ograniczone VARCHAR, LONGTEXT, BIGINT i TINYINT według **backend/mysql_database.py**. Pole **password** zawiera hash, nie hasło jawne. **expires** to czas Unix w sekundach; czas historii **at** jest w milisekundach.

Fizyczne FK: **sessions.user_id** i **members.user_id** do **users.id**. Relacje do wyjazdu w **records**, operacji użytkownika i pól payloadu są logiczne. Tabela **operations** backendu przechowuje wyniki już obsłużonych operacji dla idempotencji; lokalna tabela o tej samej nazwie jest kolejką oczekującą. Nie należy utożsamiać ich funkcji.

Payloady rekordów

kind	Model / dane
trip	TripData: tytuł, cel, daty, waluta, opis
member	MemberData: userId, nazwa i rola
event	EventData: tytuł, data, godzina, miejsce, opis, rank, wykonanie i reminder
task	TaskData: kategoria, assignee, termin, wykonanie i reminder
proposal	ProposalData: tytuł, miejsce i opis
vote	VoteData: proposalId i userId
expense	ExpenseData: kwota, payer, shares, data i category
budget	BudgetData: limit i categoryLimits

kind	Model / dane
settlement	SettlementData: from, to, amount, data i note
place	PlaceData: tytuł, latitude, longitude i opis
activity	ActivityData: autor, akcja, rodzaj, tytuł i czas serwera

Wyjazd ma **id=tripId**, członkostwo **id=tripId:userId**, głos **id=proposalId:userId**, budżet **id=budget:tripId**. Pozostałe rekordy otrzymują ID. Daty kalendarzowe są ISO **YYYY-MM-DD**; godziny to **HH:mm**. Kwoty to całkowite grosze/centy. Kategoria nieobecna w starszym koszcie jest interpretowana jako **other**.

Integralność i trwałość

Serwer sprawdza istnienie powiązanych osób i wyjazdu, własność głosu/spłaty, sumę udziałów oraz ograniczenia waluty. Waluty nie można zmienić po zapisaniu aktywnych kosztów, budżetu lub spłat. Usunięcie użytkownika z wyjazdu z aktywnymi powiązaniami finansowymi jest ograniczone. Zniknięcie uprawnień jest odtwarzane na urządzeniu po synchronizacji.

Zapis JSON rozszerza modele bez zmiany schematu Room; każda przyszła zmiana fizycznego schematu wymaga osobnej strategii migracji. Lokalna baza nie jest szyfrowana SQLCipher. Kopie SQLite wykonuje **backend/backup.py**; MySQL wymaga eksportu bazy na hostingu.

Biblioteki i technologie

Poniższa tabela przedstawia technologie i wersje deklarowane w konfiguracji projektu. Wersje zależności przechodnich są rozwiązywane przez Gradle.

Technologia	Deklaracja / wersja	Zastosowanie
Kotlin	Wbudowany Kotlin AGP; plugin Compose 2.2.10	Android, modele i domena
Android Gradle Plugin	9.1.1	Budowanie aplikacji
Gradle Wrapper	9.3.1	Powtarzalne uruchamianie Gradle
Android SDK	compile 36.1, target 36, min 26	Kompilacja i zgodność Android 8.0+
Java / JDK	Kod Java 17; sprawdzono JDK 21	Narzędzia budowania
Compose BOM	2024.01.00	Deklarowana platforma zależności Compose
Compose / Material 3	Wersje zarządzane grafem Gradle	Ekrany, motywy i kontrolki
Activity Compose	1.13.0	Aktywność, stan i obsługa Wstecz
Lifecycle	2.9.0	ViewModel i zbieranie stanu zgodnie z cyklem życia
Navigation Compose	2.7.7	Nawigacja wieloekranowa
Room	2.8.4	SQLite lokalne, DAO, transakcje i Flow
KSP	2.2.10-2.0.2	Generowanie implementacji Room
DataStore	1.1.7	Motyw i adres API
WorkManager	2.9.0	Okresowa synchronizacja i przypomnienia
Coroutines	1.10.2	Asynchroniczność, Flow i współbieżność
Retrofit / converter-gson	2.9.0	Klienci REST i DTO
OkHttp	4.12.0	Transport HTTP
Gson	2.11.0	Kodowanie payloadów i DTO
Android Keystore / AES-GCM	API Android	Szyfrowana sesja lokalna
Python	3.10+; lokalnie testowano 3.13	Backend REST i testy
SQLite	Biblioteka standardowa Python	Lokalny backend i testy
PyMySQL	1.2.3, wariant rsa	Adapter bazy MySQL na hostingu
Leaflet	1.9.4	Biblioteka map dołączona lokalnie w assets
OpenStreetMap	Kafelki HTTPS	Obraz mapy i atrybucja
Open-Meteo	Geocoding / forecast API	Miejscowości i 7-dniowa pogoda
JUnit	4.13.2	Testy jednostkowe Kotlin
AndroidX Test	runner 1.7.0, ext-junit 1.3.0	Testy urządzeniowe
Espresso	3.7.0	Test kompaktowego systemowego kalendarza

Technologia	Deklaracja / wersja	Zastosowanie
Compose UI Test	Graf Compose	Testy formularzy i nawigacji
ReportLab	Narzędzie generatora dokumentacji	PDF, bez zależności aplikacji od Pythona

Licencje i źródła

Leaflet ma licencję BSD-2-Clause dołączoną w **app/src/main/assets/leaflet/LICENSE**. Mapa wyświetla atrybucję OpenStreetMap. Warunki usług Open-Meteo i kafelków trzeba respektować także w projekcie bezpłatnym; nie pobieramy całych obszarów map offline. Przed komercyjną rozbudową trzeba ponownie sprawdzić warunki dostawców.

- Compose i BOM
- MVVM / architektura Android
- Room
- WorkManager
- Android Keystore
- Retrofit
- Leaflet
- Open-Meteo
- Polityka kafelków OSM

API REST - kontrakt backendu 1.2.0

Android 1.2.1 wymaga API 1.2.0 do atomowego przenoszenia pomysłów. Dotychczasowe operacje pozostają obsługiwane. Endpoint **/health** raportuje wersję backendu, a nie interfejsu.

JSON UTF-8; wszystkie endpointy poza health i uwierzytelnianiem wymagają **Authorization: Bearer TOKEN**. Odpowiedź błędu HTTP zawiera **message**. Limit treści żądania: 256 KiB. Żądania nie mają interfejsu CORS dla przeglądarki; klientem jest aplikacja Android.

Metoda	Ścieżka	Dane / wynik
GET	**/health**	**status**, **version**
POST	**/auth/register**	**name**, **email**, **password** → **token**, **user**
POST	**/auth/login**	**email**, **password** → **token**, **user**
POST	**/auth/logout**	Unieważnia bieżącą sesję
GET	**/sync**	**tripIds**, **records** dostępnych wyjazdów
POST	**/operations**	**operations** → **results**; od 1 do 100 operacji
GET	**/trips/{id}/invite**	Kod zaproszenia organizatora
POST	**/trips/{id}/invite**	Nowy kod; poprzedni traci ważność
POST	**/join**	**code** → **tripId**

Operacja

```
{
  "id": "uuid-operacji",
  "recordId": "uuid-rekordu",
  "tripId": "uuid-wyjazdu",
  "kind": "event",
  "payload": {
    "title": "Spacer", "date": "2026-10-11", "time": "09:00",
    "place": "Zakopane", "description": "", "rank": 0,
    "done": false, "reminder": false
  },
  "baseVersion": 0,
  "deleted": false
}
```

Wyniki: **applied** z aktualnym rekordem, **conflict** z wersją serwera, **forbidden** lub **invalid** z komunikatem. Wynik operacji jest niezależny od pozostałych elementów batcha; nie jest to transakcja całej paczki. Rekord ma pola **id**, **tripId**, **kind**, **payload**, **version**, **deleted**.

Rodzaje rekordów: trip, member, event, task, proposal, vote, expense, budget, settlement, place oraz generowane przez serwer activity. Wyjazd ma **id=tripId**. Uczestnik ma **id=tripId:userId**. Głos ma **id=proposalId:userId**. Wydatki używają całkowitych kwot w groszach/centach i mapy **shares** od identyfikatora użytkownika do jego udziału.

member powstaje wyłącznie po utworzeniu wyjazdu lub dołączeniu przez zaproszenie. API operacji dopuszcza tylko usunięcie uczestnika przez organizatora. Nie ma endpointu zmiany właściciela.

Atomowe przeniesienie pomysłu do planu

Operacja **kind=convert** ma **recordId** równy ID pomysłu, **baseVersion** jego wersji oraz payload EventData. W jednej transakcji serwer waliduje punkt planu, zmienia rodzaj rekordu z **proposal** na **event**, usuwa aktywne głosy i zapisuje historię. Identyfikator pozostaje ten sam. **convert** jest typem operacji, nie osobnym rodzajem rekordu bazy.

```
{
  "id": "uuid-operacji",
  "recordId": "uuid-pomyslu",
  "tripId": "uuid-wyjazdu",
  "kind": "convert",
  "payload": {
    "title": "Spacer", "date": "2026-12-02", "time": "10:00",
    "place": "", "description": "", "rank": 0,
    "done": false, "reminder": false
  },
  "baseVersion": 1,
  "deleted": false
}
```

Równoczesna konwersja przez drugą osobę zwraca konflikt z istniejącym punktem zamiast tworzyć duplikat. Odrzucenie danych lub błąd transakcji pozostawia pomysł i głosy. Dla pomysłu utworzonego wyłącznie lokalnie **baseVersion=0** tworzy od razu punkt planu. Ponowienie przyjętej operacji jest idempotentne. Dalsza edycja przed synchronizacją zachowuje operację konwersji. Wybór lokalnej wersji wobec nowszego pomysłu ponawia **convert**; wobec istniejącego punktu jest zwykłą edycją **event**.

Payloady obsługiwane przez Android 1.2

- **expense**: dodatkowe opcjonalne **category**: transport, accommodation, food, activities, shopping, other. Starszy rekord bez pola traktowany jest jako other.
- **budget**: stały **recordId=budget:tripId**, **limit** i mapa **categoryLimits** (kwoty całkowite w groszach/centach). Jeden budżet wyjazdu; zapis/usunięcie/przywrócenie tylko przez organizatora. Suma limitów kategorii ≤ limit całkowity.
- **settlement**: **from**, **to**, **amount**, **date**, **note**. Nadawca musi być zalogowanym użytkownikiem i nie może zmienić właściciela istniejącej spłaty. Tylko nadawca edytuje i usuwa. Data nie może być z przyszłości. To deklaracja, bez potwierdzenia bankowego.
- **place**: **title**, **latitude**, **longitude**, **description**. Zakresy współrzędnych [-90,90] i [-180,180].
- **activity**: **actorId**, **actorName**, **action**, **recordKind**, **title**, **at** (czas serwera w ms). Powstaje wyłącznie na serwerze, w transakcji zapisu. Ponowienia i konflikty nie tworzą powtórnych wpisów. Nie przyjmuje operacji klienta i nie zawiera sekretów zaproszeń.

Waluta jest blokowana po dodaniu aktywnych wydatków, budżetu lub spłat. Usunięcie uczestnika z aktywnymi spłatami wymaga ich wcześniejszego usunięcia przez nadawcę.

Zewnętrzne publiczne API

Android używa oddzielnych klientów Retrofit bez tokenu TripSync:

- **https://geocoding-api.open-meteo.com/v1/search?name=...&count=8&language=pl** - miejscowości.
- **https://api.open-meteo.com/v1/forecast?latitude=...&longitude=...&daily=weather_code,temperature_2m_max,temperature_2m_min,precipitation_probability_max&timezone=auto&forecast_days=7** - prognoza.
- **https://tile.openstreetmap.org/{z}/{x}/{y}.png** - kafelki, standardowy cache WebView, identyfikujący User-Agent TripSync/<wersja aplikacji>, widoczna atrybucja, brak masowego pobierania.

Źródła: Open-Meteo, geokodowanie, polityka kafelków. Darmowe Open-Meteo jest przeznaczone do niekomercyjnego użycia i ma limity oraz brak SLA. Współrzędne wybranego miejsca są przekazywane tym dostawcom po użyciu mapy/prognozy; lokalizacja telefonu nie jest pobierana.

Odpowiedzi HTTP

Status	Znaczenie
200	Sukces, także batch z wynikami poszczególnych operacji
400	Błędny JSON lub dane żądania
401	Brak tokenu, błędna lub wygasła sesja
403	Brak wymaganych uprawnień
404	Nieznany endpoint lub zasób
429	Limit prób rejestracji/logowania
503	Chwilowa niedostępność bazy lub I/O
500	Inny błąd serwera, bez ujawniania szczegółów

Status wyniku operacji **conflict** nie jest osobnym HTTP 409: batch odpowiada 200, a klient odczytuje **results[].status**. Autoryzacja całego żądania jest sprawdzana przed obsługą paczki.

Przykład publicznego endpointu bez sekretów:

```
curl https://tripsync.wisp.uno/health
```

Dokładny model rekordu i schemat bazy opisuje **baza-danych.md**.

Instalacja i uruchomienie

Android Studio na Macu i pozostałych systemach

1. Sklonuj repozytorium przez GitHub Desktop lub Git. Otwórz cały katalog z **settings.gradle.kts**, nie sam folder **app**.
2. W Android Studio zainstaluj platformę Android SDK 36.1 i Build Tools 36.0.0.
3. Ustaw Gradle JDK 17 lub 21; lokalnie zweryfikowano JDK 21 z Android Studio. Używaj dołączonego Gradle Wrapper 9.3.1.
4. Android Studio utworzy lokalny **local.properties** z **sdk.dir**. Nie wysyłaj go do Git.
5. Wybierz **File → Sync Project with Gradle Files**. Przy pierwszym pobraniu zależności potrzebny jest internet.
6. Wybierz emulator lub telefon z Androidem 8.0+ i konfigurację **app**, następnie **Run**.

Emulator używany w testach: Android 10 / API 29, ARM64. Minimalny API aplikacji to 26; nie należy go zmieniać tylko po to, aby uruchomić aplikację na nowszym emulatorze.

Gradle i pliki wynikowe

```
./gradlew :app:assembleDebug
```

Na macOS pliki powstają w **~/.gradle/tripsync-builds/<identyfikator-katalogu>/app/**. Przeniesienie projektu do innego katalogu nada osobny identyfikator. Na innych systemach jest to **app/build/**.

Dla dokładnie znanej lokalizacji:

```
./gradlew -PtripsyncBuildRoot=/tmp/tripsync-build :app:assembleDebug
```

APK: **/tmp/tripsync-build/app/outputs/apk/debug/app-debug.apk**. W przypadku konfliktów synchronizacji Dokumentów nie edytuj plików **.class**; są generowane. W tym projekcie lokalizacja poza Dokumentami zapobiega ich użyciu w kolejnych kompilacjach. Nie zmieniaj źródeł na podstawie dekompilowanych klas.

Pełny skrypt weryfikacji i kopiowania APK:

```
sh scripts/build.sh
```

Skrypt używa katalogu tymczasowego lub zmiennej **TRIPSYNC_BUILD_ROOT** i zapisuje końcowy APK w **output/apk/TripSync-debug.apk**.

Instalacja APK na telefonie

Zainstaluj APK z lokalnej paczki lub artefaktu udanego GitHub Actions. Przy instalacji ręcznej Android może wymagać jednorazowej zgody dla źródła pliku. Przy urządzeniu USB:

```
adb install -r output/apk/TripSync-debug.apk
```

W przypadku kilku urządzeń wybierz jedno: **adb -s IDENTYFIKATOR install -r ...**. Aktualizacja zachowuje dane, jeśli pakiet i podpis są zgodne. APK debug z innego komputera może mieć inny klucz: nie odinstalowuj aplikacji z ważnymi niewysłanymi danymi wyłącznie po to, aby obejść błąd podpisu.

Demo i prawdziwe konta

Wypróbuj bez konta tworzy lokalny przykład. Demo nie wysyła danych do serwera. Wylogowanie czyści lokalne dane i niewysłane zmiany; ponowne wejście do demo odtwarza przykład.

Nowa instalacja używa **https://tripsync.wisp.uno/**. Dotychczas zapisany adres jest zachowany. Adres można zmienić w ustawieniach po wylogowaniu, przed utworzeniem konta. Każda osoba korzysta z własnego konta na tym samym serwerze; nie używa hasła bazy MySQL.

Własny backend lokalny

Python 3.10+. Wariant SQLite korzysta z biblioteki standardowej:

```
python3 backend/server.py --db /tmp/tripsync-local.sqlite
```

Domyślny port to 8080. W osobnym terminalu:

```
curl http://127.0.0.1:8080/health
```

Odpowiedź bieżącego backendu: **{"status":"ok","version":"1.2.0"}**. Numer wersji API nie jest numerem wersji interfejsu Android 1.2.1; API 1.2.0 dodaje atomową operację convert wymaganą przez aplikację 1.2.1.

Ustaw w emulatorze debug **http://10.0.2.2:8080/**. Telefon przez USB może użyć:

```
adb reverse tcp:8080 tcp:8080
```

Wtedy adres aplikacji to **http://127.0.0.1:8080/**. Lokalny HTTP jest dopuszczony tylko w debug dla **10.0.2.2**, **127.0.0.1** i **localhost**. Udostępnienie innym urządzeniom wymaga HTTPS z zaufanym certyfikatem; instrukcja hostingu: wispbyte.md.

Diagnoza najczęstszych problemów

Objaw	Sprawdzenie
Nie ma konta po wejściu do demo	Demo jest odrębne i lokalne; użyj rejestracji na wybranym API
Brak synchronizacji	Adres API, internet, stan kolejki oraz komunikat sesji/konfliktu
defined multiple times i nazwy **2.class**	Zsynchronizuj aktualny Gradle; generowane pliki na Macu powinny być poza Dokumentami
Nie działa MySQL	Sprawdź użytkownika bazy, host, hasło i dostęp; używaj panelu Wispbyte, nie ustawień Androida
.env zgłasza numer wiersza	Każdy wpis ma formę **NAZWA=wartość**; samej nazwy bez **=** parser nie zaakceptuje
Mapa lub pogoda nie ładuje się	Dostęp do zewnętrznych HTTPS; zapisane miejsca i cache pogody mogą pozostać dostępne
Brak powiadomienia	Zgoda od Androida 13, włączone przypomnienie oraz ograniczenia pracy w tle

Bezpieczeństwo i obsługa błędów

Android

Sesja jest szyfrowana AES-GCM. Klucz powstaje w Android Keystore; zaszyfrowane dane i IV znajdują się w prywatnych SharedPreferences. Hasło logowania nie jest zapisywane przez rememberSaveable. Baza Room znajduje się w prywatnym katalogu aplikacji, bez dodatkowego szyfrowania SQLCipher. Backup aplikacji i transfer danych są wyłączone w konfiguracji.

W release obowiązuje HTTPS. Debug dopuszcza HTTP tylko dla lokalnych adresów testowych. Zmiana serwera jest możliwa bez aktywnej sesji, aby nie wysłać tokenu do innego API. Wylogowanie czyści bazę, kolejkę, konflikty i sesję lokalną; niewysłane zmiany zostaną utracone.

Backend

Hasła: PBKDF2-HMAC-SHA256, losowa sól, 260 000 iteracji i porównanie stałoczasowe. Token sesji zawiera 48 losowych bajtów przed kodowaniem; backend przechowuje jego hash SHA-256. Sesja wygasa po 7 dniach. Limiter rejestracji/logowania: 30 prób na adres IP w 5 minut, w pamięci pojedynczego procesu.

Każda operacja sprawdza sesję, dostęp do wyjazdu, rolę, payload i wersję. Organizator zarządza wyjazdem, budżetem, zaproszeniami i uczestnikami. Użytkownik głosuje za siebie i edytuje wyłącznie własne spłaty. Parametry SQL są przekazywane jako parametry, nie przez sklejanie danych użytkownika z zapytaniem.

Hosting i dane

Hasło MySQL znajduje się wyłącznie w Environment albo **.env** serwera. **.gitignore** wyklucza sekrety, bazy, kopie i klucze podpisu. HTTPS aplikacji zabezpiecza kanał Android → API; nie zapewnia automatycznie TLS kanału API → MySQL. Ustawienie **TRIPSYNC_MYSQL_SSL_CA** włącza weryfikację TLS bazy, jeśli hosting udostępnia odpowiedni certyfikat.

Nagłówki adresu klienta są akceptowane tylko od jawnie zaufanych proxy. Kopie bazy muszą być przechowywane poza tym samym serwerem; **backend/backup.py** obsługuje SQLite, a MySQL wymaga eksportu narzędziem hostingu lub **mysqldump**.

Mapa i zewnętrzne API

Leaflet jest lokalnym zasobem. WebView nie ma mostka JavaScript do Androida, blokuje dostęp do plików/treści, mieszany HTTP i niepożądaną nawigację. Nazwy markerów są kodowane przez Gson oraz wstawiane jako textContent. CSP ogranicza źródła. Android nie pobiera GPS telefonu; współrzędne miejsca są przekazywane dostawcy mapy i prognozy. Klient Open-Meteo nie otrzymuje tokenu TripSync.

Obsługa błędów

Walidacja daje opis pola lub reguły przed zapisem. Błędy HTTP są mapowane z komunikatu serwera, a brak sieci pozostawia lokalne dane i kolejkę. Konflikt nie jest nadpisywany automatycznie. Mapa pokazuje błąd i ponowienie; pogoda zachowuje cache z datą. Backend zwraca ujednolicone JSON **message**, bez ujawniania sekretów konfiguracji.

Ograniczenia

To podstawowe zabezpieczenia projektu dydaktycznego, bez przeprowadzonego audytu bezpieczeństwa. Nie ma odzyskiwania hasła, weryfikacji e-mail, samodzielnego usuwania konta ani monitoringu produkcyjnego. Urządzenie offline może nadal wyświetlać kopię danych po odebraniu dostępu na serwerze, aż do udanej synchronizacji. Pełna lista: **ograniczenia.md**.

Testowanie i wyniki wersji 1.2.1

Data wykonania: **06.11.2026**. Android: **1.2.1**, versionCode **4**. Backend: **API 1.2.0**.

Wyniki końcowe

Grupa	Liczba testów	Niepowodzenia
Backend Python	45	0
Jednostkowe Android / Kotlin	23	0
Android: UI, integracja REST i cache pogody	31	0
Razem, różne testy	99	0

Dodatkowo powtórzono pięć testów formularzy oraz wybór daty i godziny na ekranie 320 dp ze skalą czcionki 1.3. Powtórzenia nie zwiększają liczby 99. Budowanie debug APK i APK testowego zakończyło się powodzeniem. Lint uruchomiony offline zgłosił 0 błędów i 1 ostrzeżenie GradleDependency o dostępnej aktualizacji Lifecycle. Wynik offline nie jest pełnym sprawdzeniem aktualności wszystkich bibliotek.

Pierwszy przebieg urządzeniowy ujawnił dwie niezgodności asercji z zachowaniem klawiatury i przewijania. Po dopasowaniu przebiegu testów powtórzono pięć testów formularzy oraz cały zestaw 31 testów; oba końcowe przebiegi zakończyły się bez błędów. Log pierwszego przebiegu pozostaje w raportach.

Środowisko i materiały wynikowe

Emulator Android 10 / API 29, ARM64, 1080 x 2400 px, 420 dpi; test kompaktowy: 840 x 1470 px, skala czcionki 1.3. Użyto osobnego tymczasowego AVD. Kompilacja: SDK 36.1, AGP 9.1.1, Gradle 9.3.1, JDK 21. Backend testowy: Python 3.13.5 i izolowana lokalna SQLite; bez kont na publicznym serwerze.

- Wyniki JSON.
- Raporty: logi backendu, budowania, przebiegu urządzeniowego i kompaktowego; JUnit XML i Lint HTML/XML/TXT.
- Zrzuty interfejsu: 18 obrazów z wersji 1.2.1.
- Raporty i screeny wersji 1.2 oraz 1.1 są archiwalne.

Testy jednostkowe Android

Odczyt i formatowanie kwot, precyzja, granice i przepełnienie; podział groszy, udziały, salda i przelewy, w tym 200 deterministycznych zestawów rozliczeń. Spłaty częściowe, pełne i nadpłaty; budżety, współrzędne oraz daty i godziny w różnych strefach i podczas zmiany czasu. Nowe przypadki sprawdzają zerowy limit kategorii, odrzucanie ujemnych i zbyt dużych kwot, nadmiernej precyzji oraz zachowanie wymogu dodatniego wydatku.

Testy backendu

Konta, hashowanie, sesje, uprawnienia, zaproszenia, idempotencja, konflikty, tombstone i głosy; koszty, kategorie, budżety, spłaty i historia. Kopia SQLite, proxy, WSGI, kontrakt HTTP, parser Environment i adapter MySQL.

Osiem nowych testów konwersji obejmuje pojedynczą transakcję i usunięcie głosów, wyścig dwóch użytkowników, błędny termin punktu, rollback po awarii bazy, idempotentne ponowienie, nowszą wersję pomysłu, pomysł utworzony wyłącznie offline oraz brak uprawnień i usunięte źródło. Równoczesna konwersja pozostawia dokładnie jeden punkt planu.

Testy adaptera MySQL korzystają ze sterownika zastępczego. Nie wykonywano regresji na rzeczywistej bazie hostingu Wispbyte.

Testy urządzeniowe

CRUD wydatków, salda, formularze, głosowanie, budżet, spłaty, miejsca, historia, kalendarz i zegar, zachowanie szkicu po odtworzeniu aktywności oraz stan filtrów. Sprawdzono ekranowe potwierdzenie odrzucenia zmian w budżecie, miejscu i spłacie, anulowanie i zatwierdzenie usuwania budżetu oraz zapis limitu 0. Testy cofania uwzględniają zamknięcie klawiatury przed systemowym Wstecz.

Integracja REST używa dwóch niezależnych repozytoriów i kont: kolejka, synchronizacja, konflikt z wyborem obu wersji, usunięcie i cofnięcie dostępu. Nowe przypadki obejmują konwersję przez dwie osoby, ponowienie wobec nowszego pomysłu, dalszą edycję konwersji offline, usunięcie oczekującej konwersji, anulowanie przypomnienia przy przyjęciu wersji serwera i natychmiastowe przekazanie przypomnienia w demo. Cache pogody sprawdzono po błędzie sieci.

Powtórzenie testów

```
python3 -m unittest discover -s backend -v
./gradlew :app:testDebugUnitTest :app:lintDebug
```

Na potrzeby testów urządzeniowych uruchom dedykowany emulator i lokalny backend:

```
python3 backend/server.py --db /tmp/tripsync-test.sqlite
./gradlew :app:connectedDebugAndroidTest
```

Testy wylogowują aplikację i odtwarzają demo, dlatego wymagają oddzielnego urządzenia testowego. Integracja używa adresu http://10.0.2.2:8080/. Przy kilku urządzeniach ustaw ANDROID_SERIAL. Każdy test tworzy odrębne konta; przy częstych pełnych powtórzeniach należy zrestartować izolowany backend testowy, aby limit logowań nie zakłócał przebiegu.

Granice weryfikacji

Nie wykonano pełnej macierzy nowszych wersji Androida, testów fizycznego telefonu i tabletu, audytu TalkBack ani testów obciążenia i bezpieczeństwa. Test odtworzenia aktywności nie jest testem utraty procesu. WorkManager podlega ograniczeniom systemu; sprawdzenie przekazania przypomnienia nie gwarantuje dokładnej godziny jego dostarczenia.

Ograniczenia aplikacji

Wersja Android 1.2.1, backend API 1.2.0.

Granice bieżącej aplikacji

- Jedna waluta na wyjazd: PLN, EUR albo USD. Nie ma przeliczania kursów.
- Maksymalnie 100 uczestników. Rozliczenia zachowują dokładność grosza/centa. Algorytm tworzy co najwyżej n-1 przelewów dla n osób z niezerowym saldem, ale nie gwarantuje globalnie najmniejszej liczby przelewów.
- Spłata jest deklaracją nadawcy; aplikacja nie potwierdza jej w banku i nie wykonuje płatności.
- Synchronizacja pobiera pełny snapshot, jest ręczna lub okresowa przez WorkManager, bez aktualizacji w czasie rzeczywistym. Usunięte uprawnienia są egzekwowane na serwerze natychmiast; urządzenie offline zachowuje wcześniej pobraną kopię do udanej synchronizacji.
- Okresowa synchronizacja i lokalne przypomnienia podlegają ograniczeniom Androida. Nie są alarmami dokładnymi.
- Wylogowanie czyści lokalną bazę i niewysłane zmiany. Demo nie jest eksportowane do prawdziwego konta.
- Mapa potrzebuje internetu do kafelków. Brak map offline i GPS telefonu. Wyszukiwanie obejmuje miejscowości; inne punkty można podać współrzędnymi.
- Prognoza obejmuje najbliższe 7 dni, nie dowolny termin przyszłego wyjazdu. Cache jest oznaczany datą.
- Historia pokazuje 200 najnowszych wpisów; brak paginacji i cofania zmian.
- Backend działa jako jeden proces z serializowanymi zapisami; nie zweryfikowano wieloprocesowego wdrożenia ani obciążenia.
- Brak odzyskiwania hasła, weryfikacji e-mail i samodzielnego usuwania konta. Brak czatu, załączników, skanowania paragonów, QR, szablonów oraz eksportu PDF/ICS z poziomu aplikacji. Udostępnianie tekstowego planu już działa.
- Nie ma płatności, premium, donate ani reklam. Zakres pozostaje projektem na zajęcia Android.

Materiały do oddania - TripSync

Wykonawcy: Brajan Sieroń, Damian Panońko, Patryk Dawczak. **Wersja aplikacji:** 1.2.1, kod wersji 4. **Data dokumentacji:** 06.11.2026.

Wykaz wymaganych materiałów

Wymagany element	Materiał
Opis celu i zakresu aplikacji	Cel, zakres i wymagania oraz rozdział PDF
Wymagania funkcjonalne i niefunkcjonalne	Specyfikacja F01-F26 i N01-N09 w wymagania.md
Diagram architektury aplikacji	Opis MVVM, diagram SVG i PDF
Diagram przypadków użycia	UML i scenariusze, diagram SVG i PDF
Diagram bazy danych	Model danych, Room, SQLite/MySQL i PDF
Zrzuty najważniejszych widoków	18 zrzutów w galerii, katalogu screenshots-1.2.1 i PDF
Biblioteki i technologie	Technologie, wersje i zastosowanie
Instrukcja instalacji i uruchomienia	Android Studio, APK i backend lokalny; Wispbyte i MySQL
Testy i ich wyniki	Raport, wyniki JSON, raporty XML/HTML/TXT i logi
Link do repozytorium Git	github.com/Yamoteusz/TripSync
Działająca aplikacja	APK debug wersji 1.2.1 w paczce do oddania

Dokument PDF i paczka

Dokument zbiorczy: TripSync_dokumentacja_projektu.pdf.

Paczka **output/TripSync_do_oddania.zip** zawiera dokument PDF, plik **TripSync.apk**, listę wykonawców, instrukcję, dokumenty Markdown, diagramy SVG, screeny oraz raporty testów. APK wymaga Androida 8.0 lub nowszego. Repozytorium zawiera kod źródłowy aplikacji i backendu.

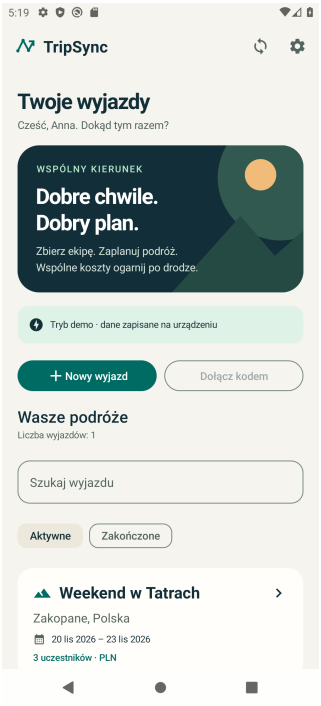
Uruchomienie prezentacji

1. Rozpakuj paczkę i zainstaluj **TripSync.apk** na telefonie lub emulatorze Android 8.0+.
2. Wybierz **Wypróbuj bez konta**, aby uruchomić lokalny przykład.
3. Do współpracy wielu osób użyj odrębnych kont na tym samym API. Adres domyślny: **https://tripsync.wisp.uno/**.
4. Pełny scenariusz pokazu znajduje się w prezentacja.md.

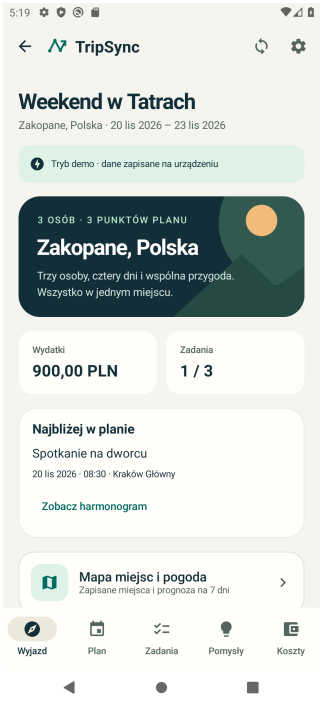
APK jest podpisany kluczem debug i służy do instalacji testowej. Wyniki obejmują 99 testów wykonanych 06.11.2026. Przed aktualizacją APK należy wdrożyć backend API 1.2.0 zgodnie z instrukcją Wispbyte.

Widoki aplikacji / 1

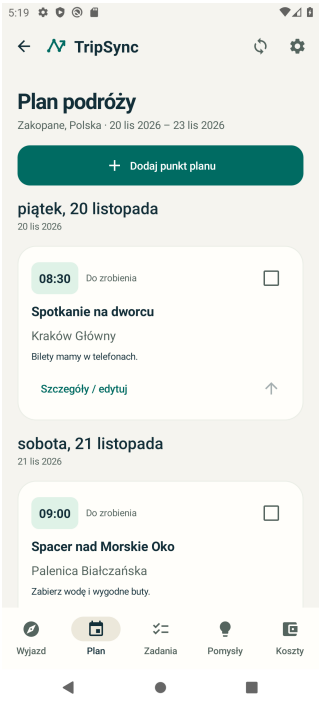
Rzeczywiste zrzuty z testów wersji 1.2.1, 06.11.2026. Dane pochodzą z demo i izolowanego środowiska testowego.



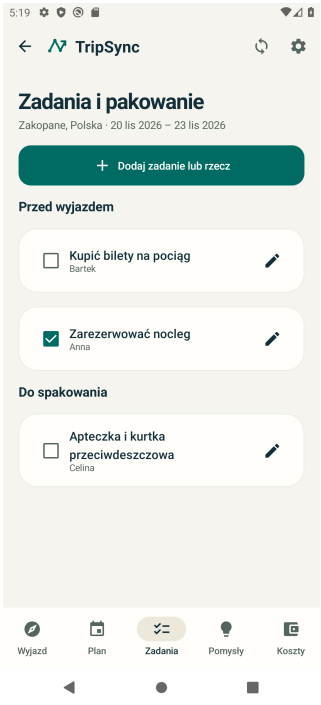
Wyjazdy



Podsumowanie



Harmonogram

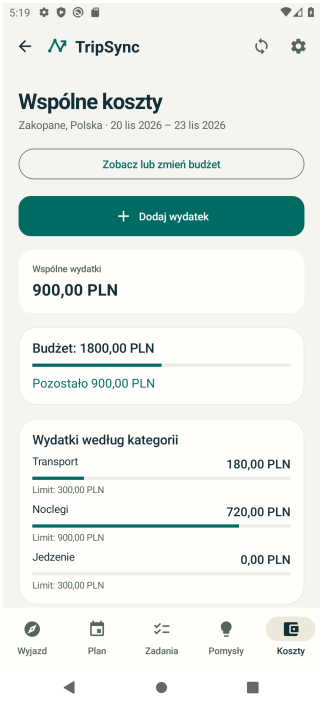


Zadania i pakowanie

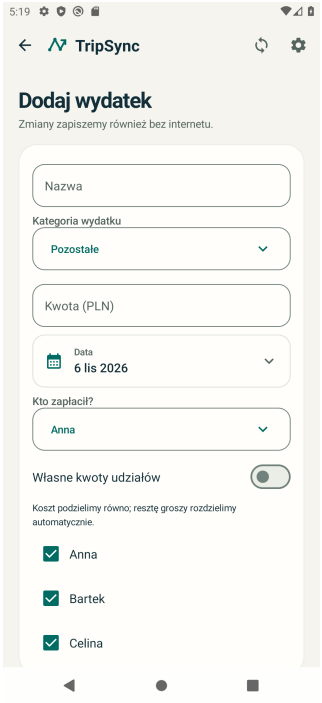
Widoki aplikacji / 2



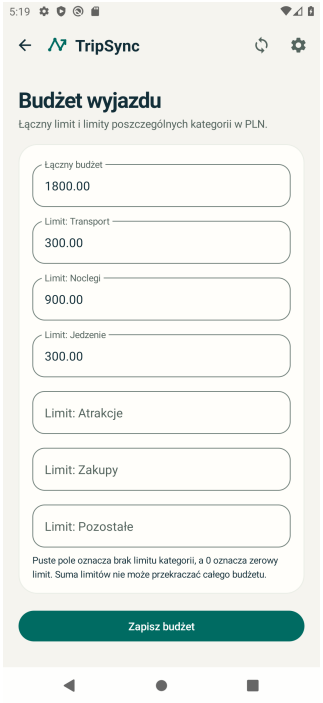
Głosowanie



Koszty i rozliczenia

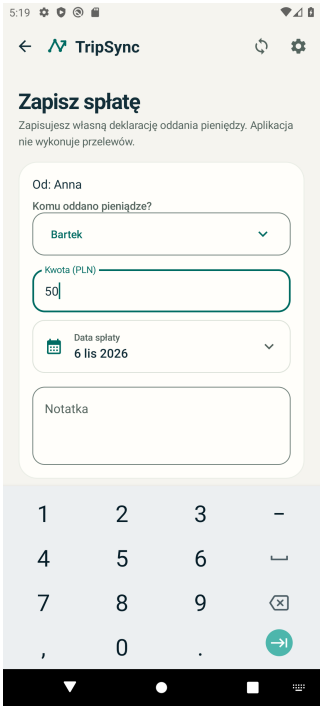


Formularz wydatku

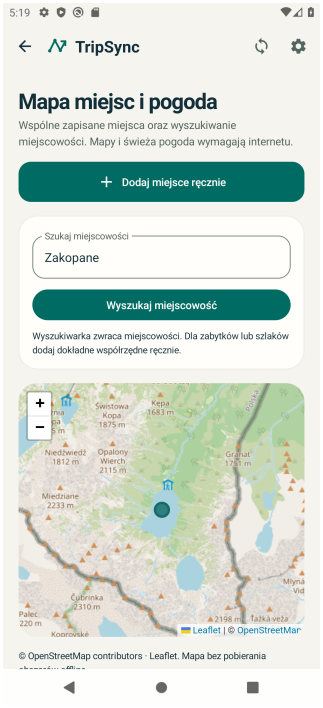


Budżet i kategorie

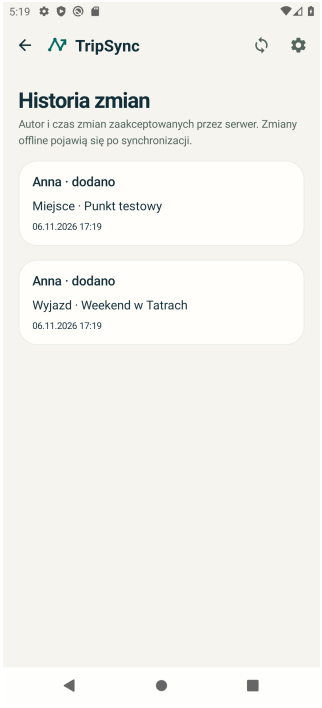
Widoki aplikacji / 3



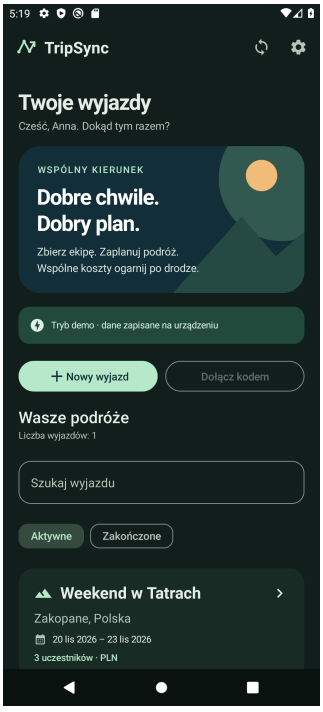
Spłata



Mapa miejsc

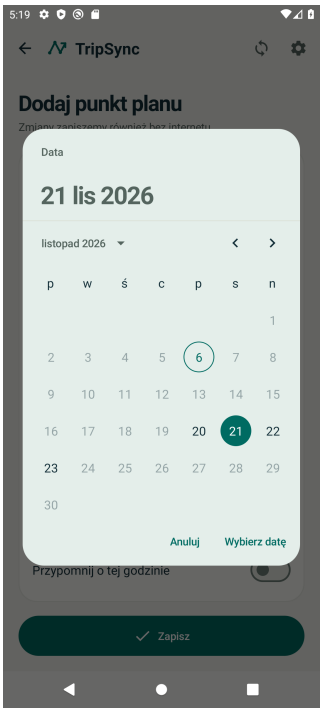


Historia zmian



Motyw ciemny

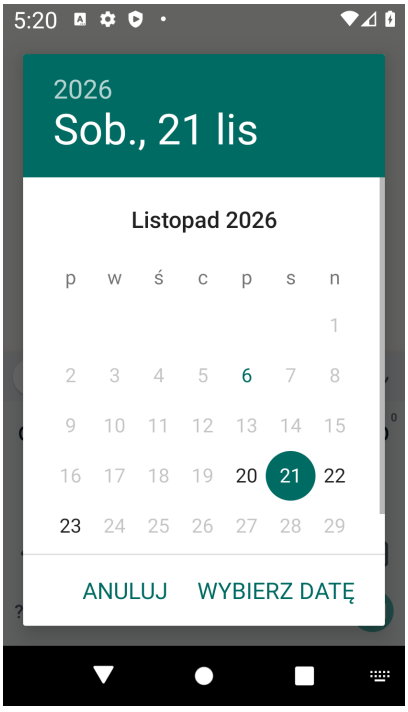
Widoki aplikacji / 4



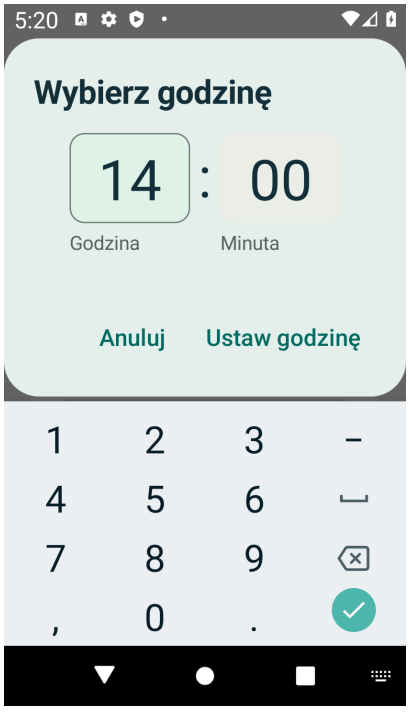
Wybór daty



Wybór godziny

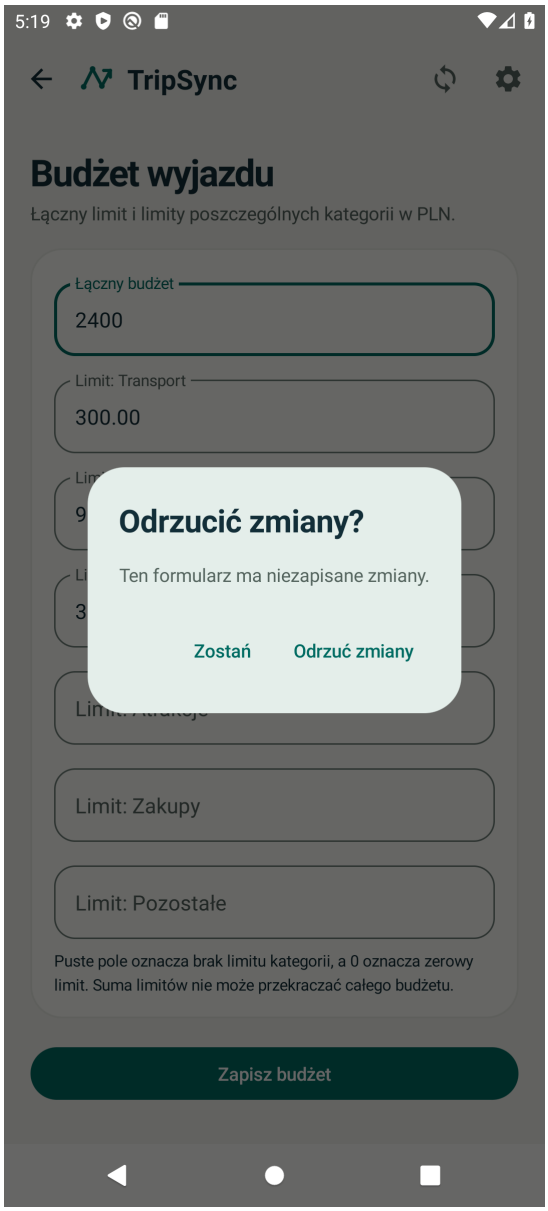


Kalendarz: mały ekran

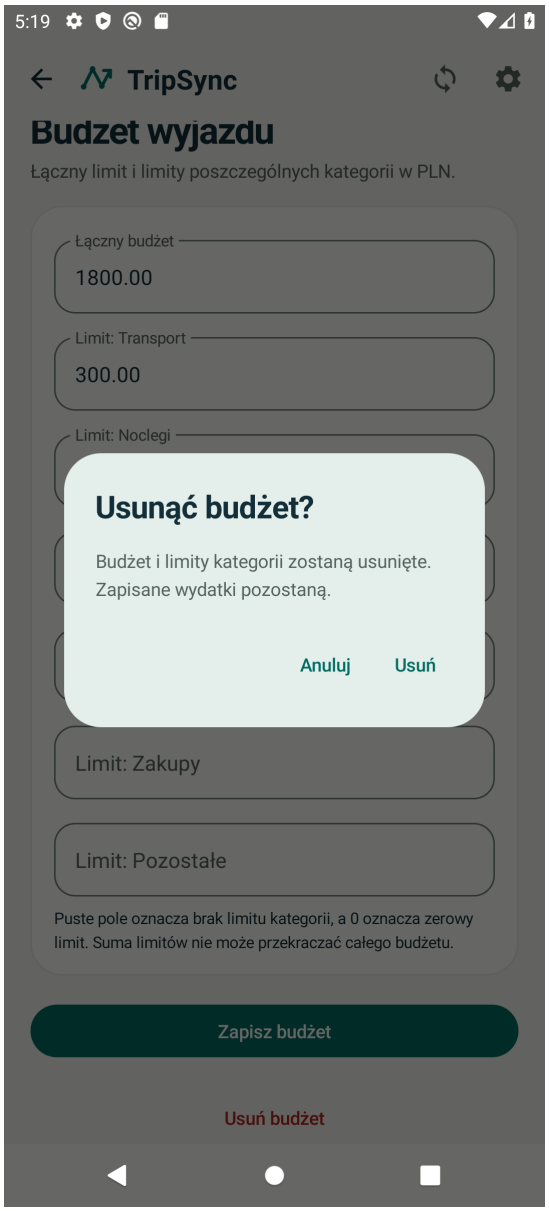


Zegar: mały ekran

Widoki aplikacji / 5



Ochrona niezapisanego formularza



Potwierdzenie usunięcia budżetu